Git & GitHub

Git & GitHub 개념 및 협업

김태민

저번에 우리는

반응형 웹 디자인

다양한 기기(스마트폰, 태블릿, 데스크톱)와 화면 크기에 맞춰
웹 페이지의 레이아웃, 콘텐츠, 기능을 최적화하는 설계 접근법

핵심 구성 요소:

- 유연한 레이아웃: 상대 단위(%, vw, vh, rem, em) 사용
- 미디어 쿼리: 화면 크기나 기기 특성에 따라 스타일 조정
- 반응형 이미지: 기기 해상도에 맞는 이미지 제공

모든 사용자에게 일관되고 최적화된 경험 제공

미디어 쿼리

CSS의 @media 규칙을 사용해 화면 크기, 기기 특성, 해상도 등 특정 조건에 따라 스타일을 적용하는 기술
반응형 웹 디자인의 핵심으로, 다양한 기기(모바일, 태블릿, 데스크톱)와 환경에 맞는 스타일링 제공

목적:

- 사용자 경험(UX) 최적화: 기기에 맞는 레이아웃 제공
- 접근성 강화: 다양한 화면 크기와 환경 지원
- 성능 개선: 조건에 따라 최적화된 리소스 로드

미디어 쿼리 - 화면크기

767px

1024px

```
@media (max-width: 767px) {
    .box {
        width: 33.33%;
    }
}
```

```
@media (min-width: 768px) and (max-width: 1023px){
    .box {
        width: 33.33%;
    }
}
```

```
@media (min-width: 1024px) {
    .box {
        width: 33.33%;
    }
}
```

max-width:
화면 너비가 특정 값 이하일 때

범위 지정

min-width:
화면 너비가 특정 값 이상일 때

상대 단위

CSS에서 요소의 크기나 간격을 상대적인 기준(부모, 뷰포트, 폰트 크기 등)에 따라 설정하는 단위

역할: 반응형 웹 디자인에서 유연하고 동적인 레이아웃 구현, 다양한 기기와 화면 크기에 적응

주요 상대 단위: %, vw, vh, em, rem

장점:

- 고정 단위(px)와 달리, 화면 크기나 폰트 크기 변화에 따라 자동 조정
- 접근성과 유지보수성 향상

모바일 퍼스트와 데스크톱 퍼스트

모바일 퍼스트:

- 기본 스타일을 모바일(작은 화면)에 맞춰 작성
- 미디어 쿼리(min-width)를 사용해 더 큰 화면에 스타일을 추가

데스크톱 퍼스트:

- 기본 스타일을 데스크톱(큰 화면)에 맞춰 작성
- 미디어 쿼리(max-width)를 사용해 작은 화면에 스타일을 조정

실습 환경

실습환경

OS : Window / Mac

브라우저 : Chrome

에디터 : VS Code https://code.visualstudio.com/

VS Code 익스텐션 : ESLint, Prettier, HTML CSS Support, HTML to CSS autocompletion, Auto Rename Tag, Auto Close Tag, htmltagwrap

Git : git bash, github 가입

실습환경

내 컴퓨터에 git이 이미 설치되어 있는지 확인

- Windows : CMD(명령 프롬프트) 열고 git 입력
- Mac : Terminal 열고 git 입력

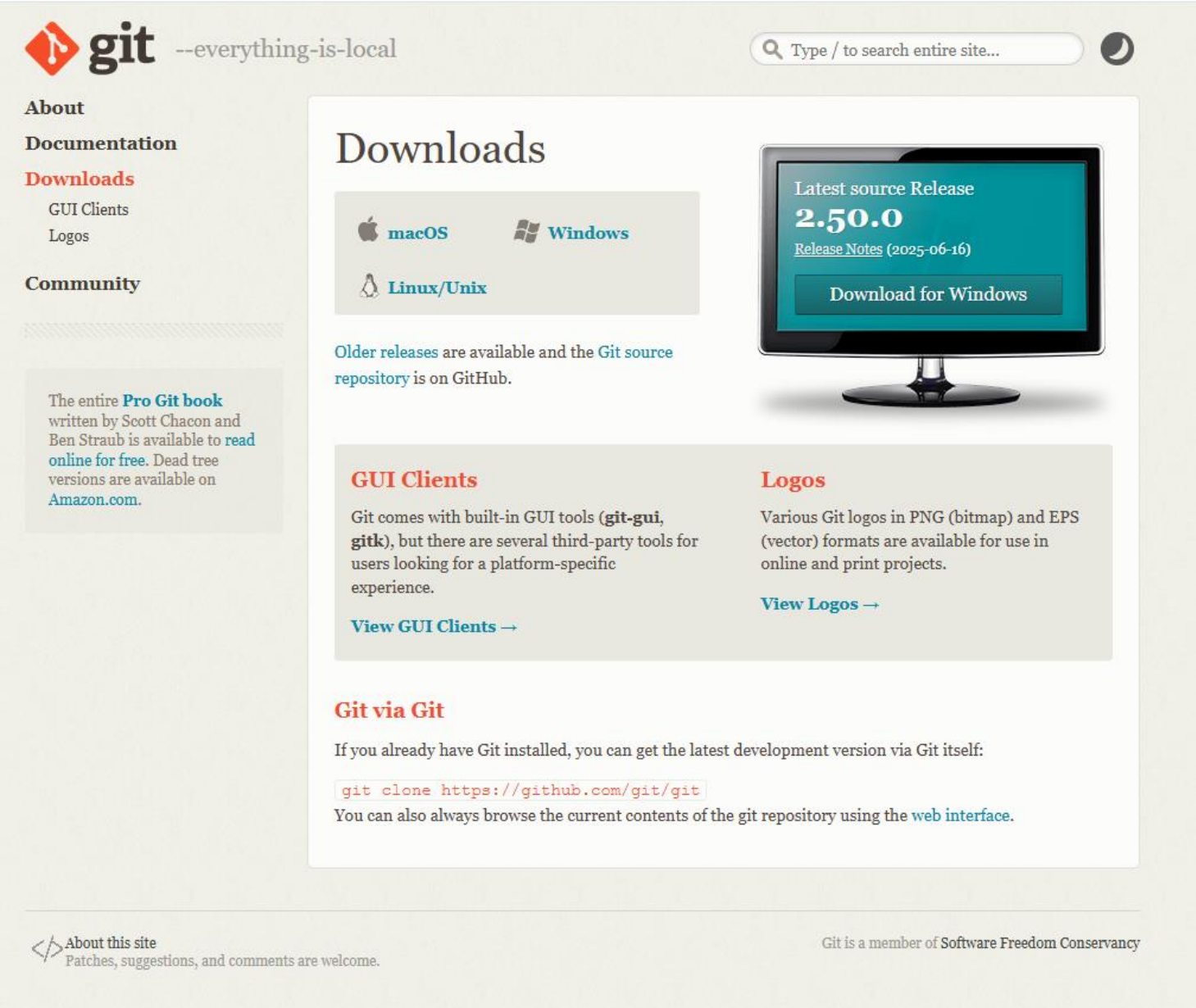
명령 프롬프트

```
   sparse-checkout   Initialize and modify the sparse-checkout

examine the history and state (see also: git help revisions)
   bisect            Use binary search to find the commit that introduced a bug
   diff              Show changes between commits, commit and working tree, etc
   grep              Print lines matching a pattern
   log               Show commit logs
   show              Show various types of objects
   status            Show the working tree status

grow, mark and tweak your common history
   branch            List, create, or delete branches
   commit            Record changes to the repository
   merge             Join two or more development histories together
   rebase            Reapply commits on top of another base tip
   reset             Reset current HEAD to the specified state
   switch            Switch branches
   tag               Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
   fetch             Download objects and refs from another repository
   pull              Fetch from and integrate with another repository or a local branch
   push              Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

C:₩Users₩DEVELOPER-01>git
```

실습환경

Git, Git Bash 설치 (CLI 환경)

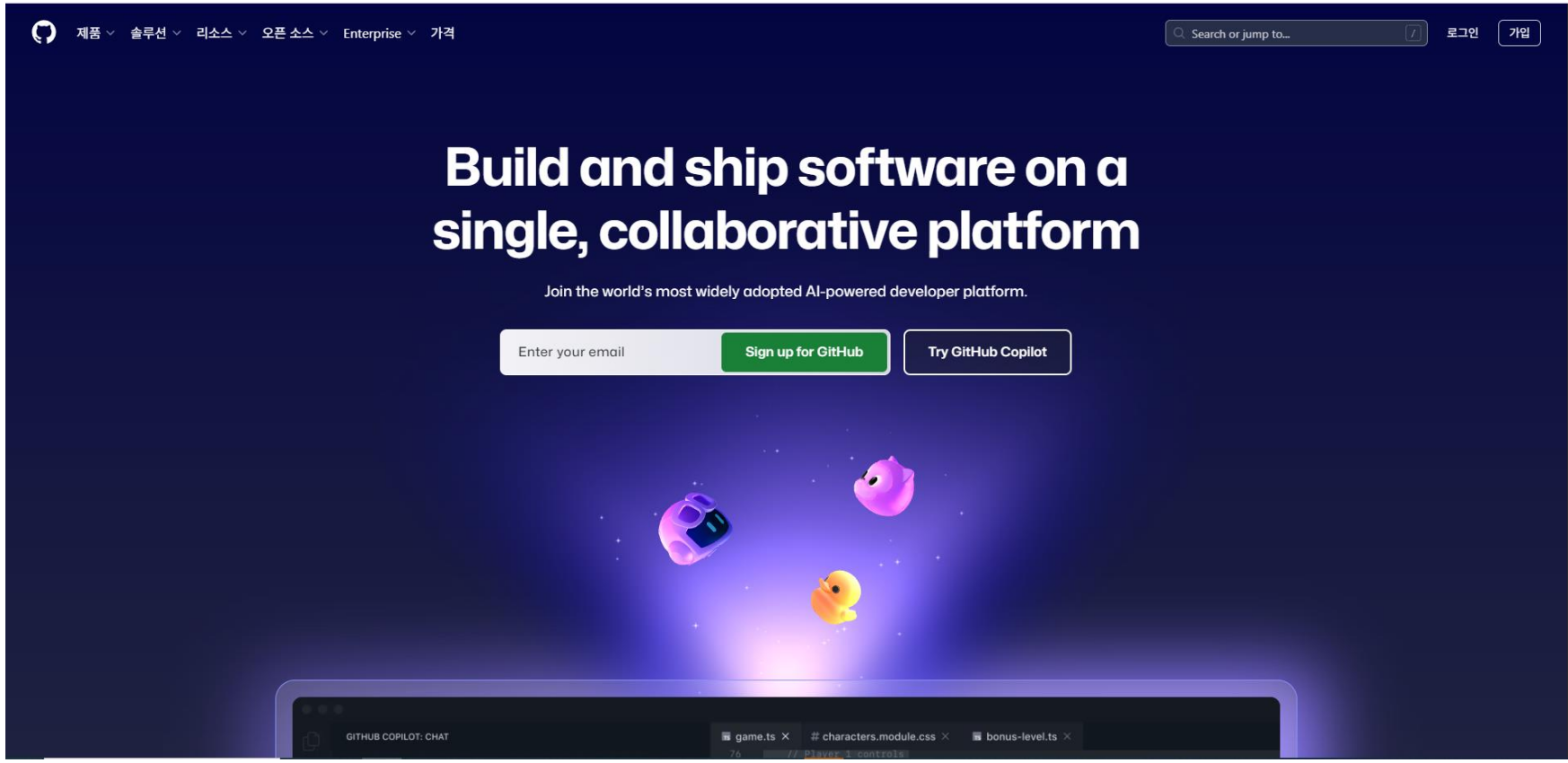
- https://git-scm.com/



실습환경

Git Hub

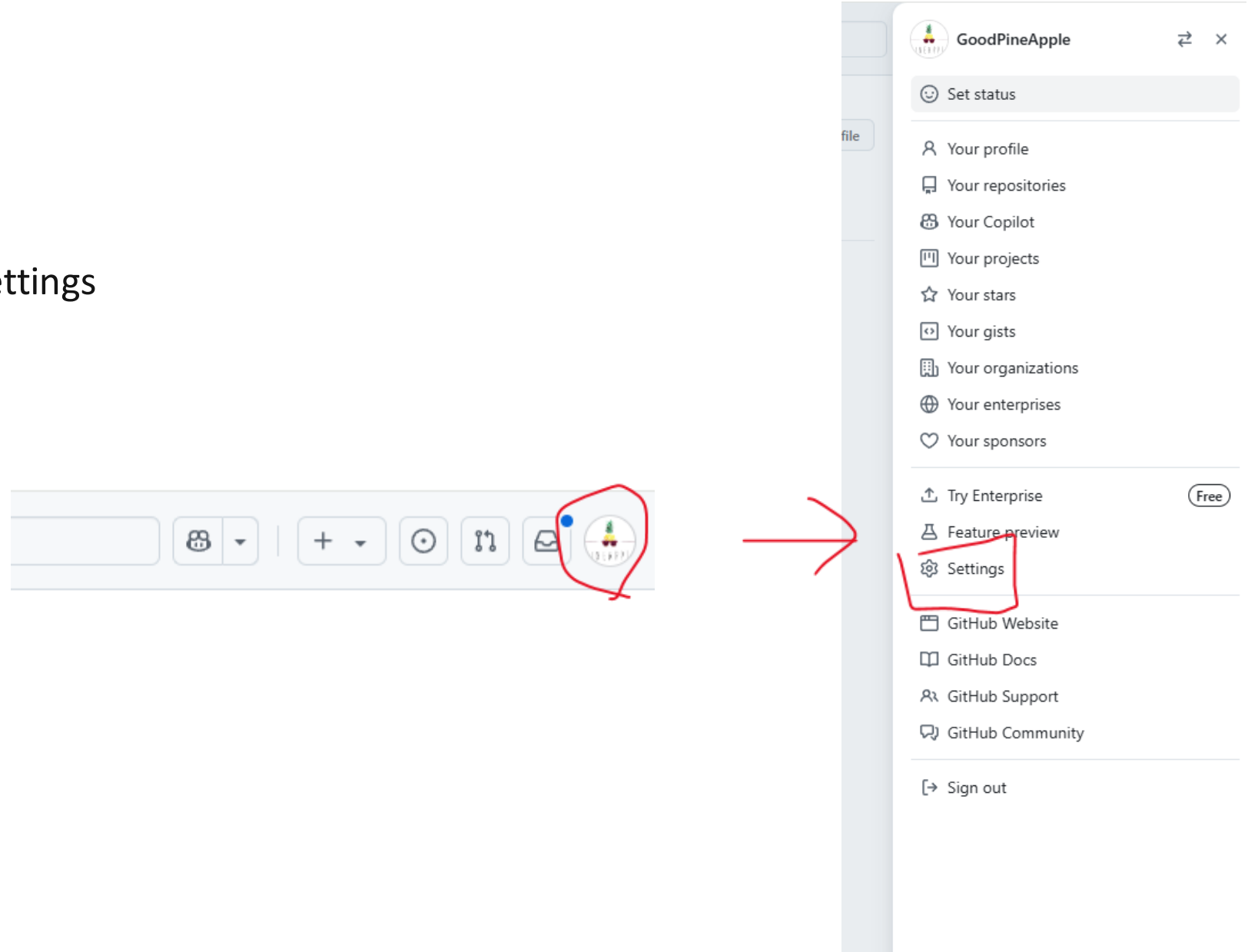
- https://github.com/



실습환경

Git Hub

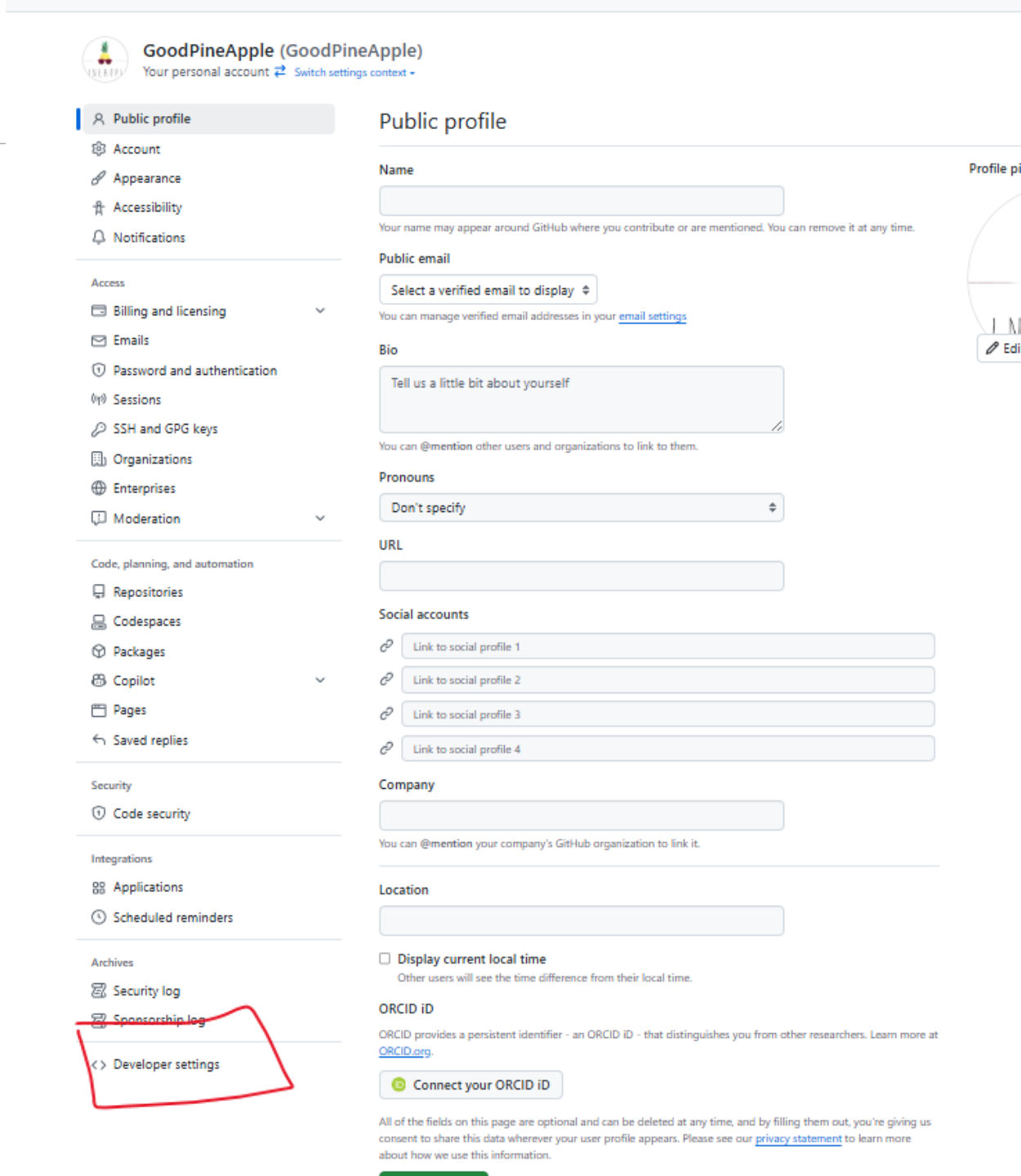
- https://github.com/
- 우측상단 프로필 – Settings



실습환경

Git Hub

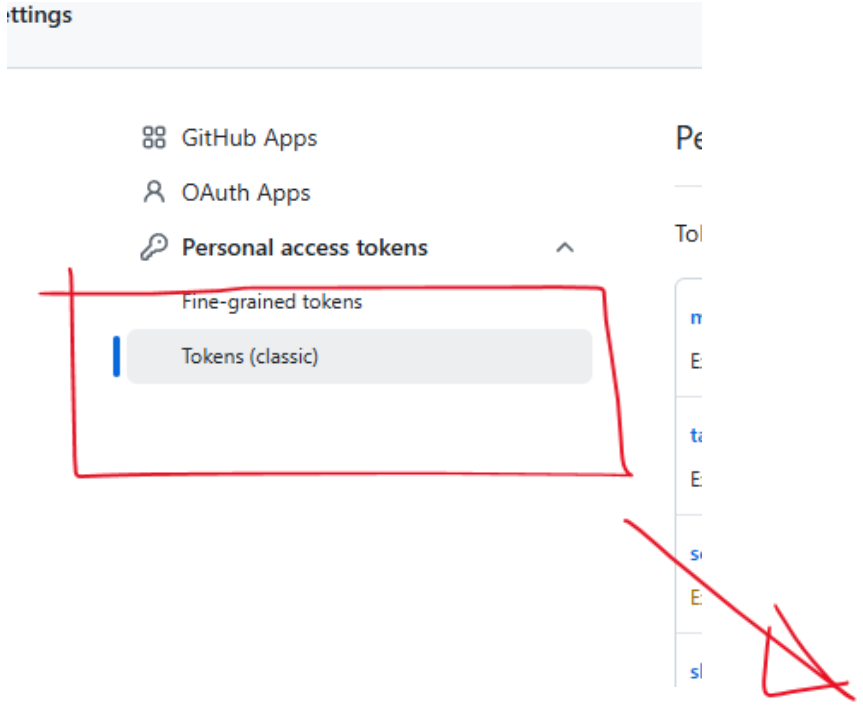
- https://github.com/
- 우측상단 프로필 – Settings
- 가장 하단 Developer Settings

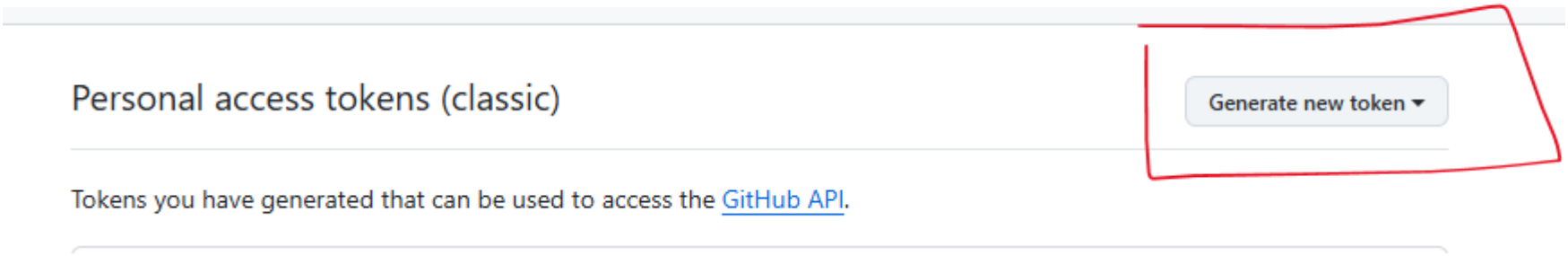


실습환경

Git Hub

- https://github.com/
- 우측상단 프로필 – Settings
- 가장 하단 Developer Settings
- Personal Access Tokens (classic)

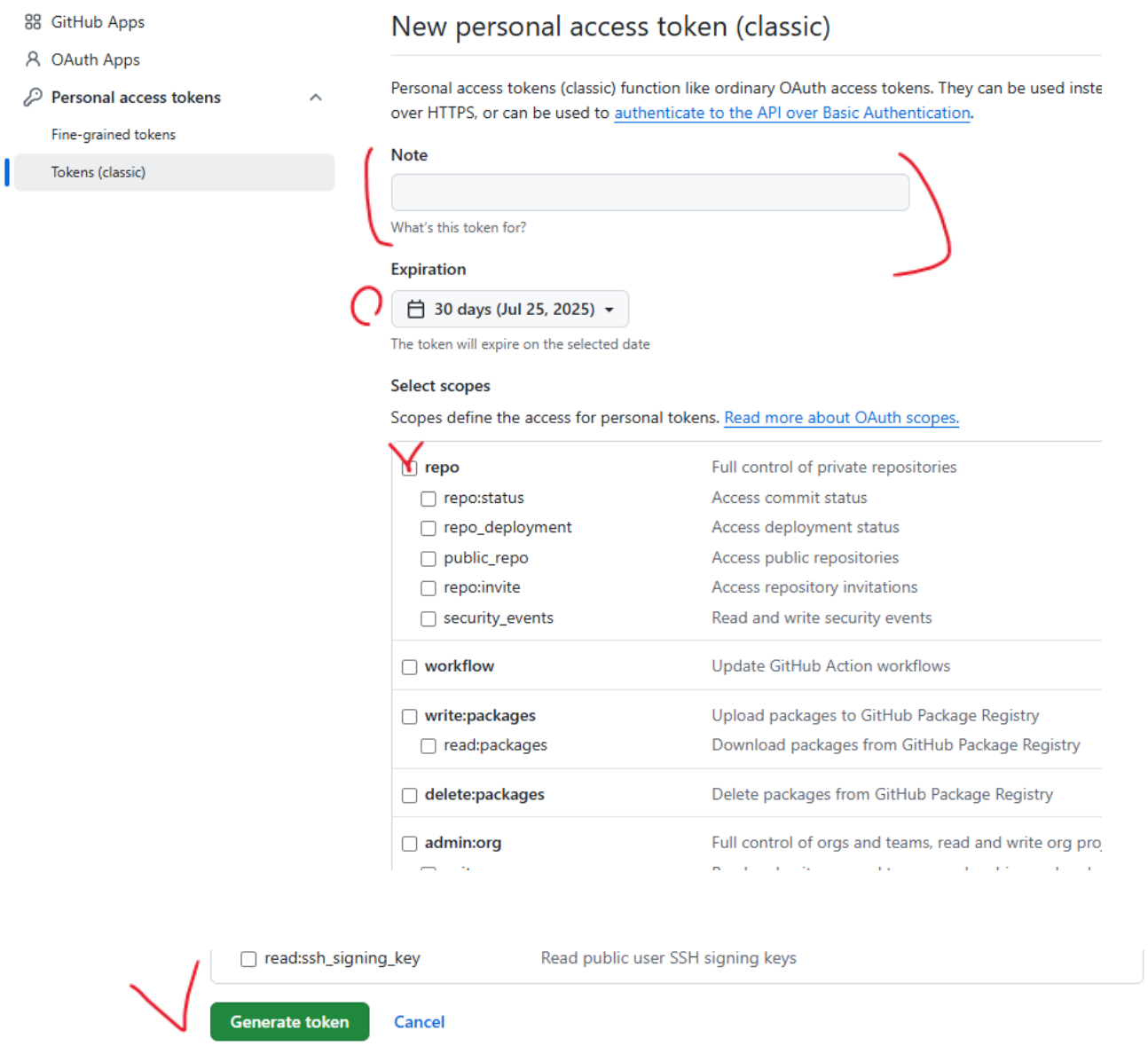




실습환경

Git Hub

- https://github.com/
- 우측상단 프로필 – Settings
- 가장 하단 Developer Settings
- Personal Access Tokens (classic)
- Repo 클릭후 생성



Git과 버전관리

Git과 버전관리

버전관리 없는 개발

"어제 수정이 최신인가요?"

"이 파일의 원본이 뭐죠?"

- 팀 작업 시 혼란
- 데이터 손실 위험

Git과 버전관리

버전관리 Version Control System

파일 변경 이력을 기록하고 관리하는 방법

1. 협업 : 여러 사람이 동시에 작업 가능
2. 복구 : 실수해도 이전 버전으로 돌아감
3. 추적 : 누가 언제 무엇을 바꿨는지 확인 가능

Git과 버전관리

개발자 01

개발자 02

Git과 버전관리

개발자 노트

·ㅁ쏘ㅅ~K ' 꼴 ' r 노트북 노트

Page01

Page02

Page03

Page01~03 생성하여
버전 'dev01'이라고 저장

개발자 노트

Git과 버전관리

□□□ 노뷘

filename: dev01_02

Page01 | Page02 | Page03

□□□ 노뷜

·ㅍㅅ~K ' 꼴 ' r 노뷜

t ~ㅋ뱐놜 | t ~ㅋ뱐놨 | t ~ㅋ뱐뇌

Page04 | Page05 | Page06

'dev01'을 다운받아서
Page04~06 생성, 'dev02'로 저장

Git과 버전관리

□□□ 노뷘

filename: dev01_02

Page01 | Page02 | Page03

Page02 수정하여
버전 'dev01_02' 라고 저장

□□□ 노뷜

·ㅍΛ~K ' 꼴 ' r 노뷜

t ~ㅋ'놘놜 | t ~ㅋ'놘놨 | t ~ㅋ'놘뇌

Page04 | Page05 | Page06

Git과 버전관리

□□□ 노뷘

filename: dev01_02

Page01 | Page02 | Page03

□□□ 노뷜

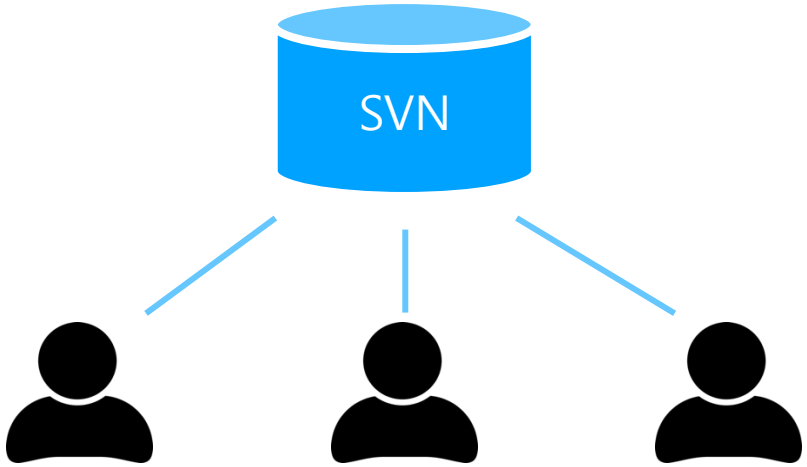
·ᄯᅡᆺᄉᆞᄏᆞ ' 꼴 ' r 노뷜

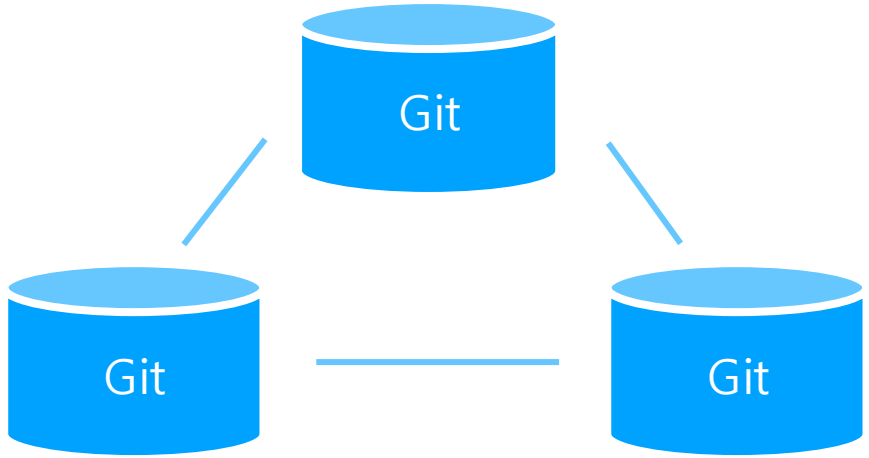
t ~ᄏᆞ뉸놜 | t ~ᄏᆞ뉸놨 | t ~ᄏᆞ뉸뇌

Page04 | Page05 | Page06

'dev01_02'와 'dev02'를 비교(diff)
Page02 수정사항을 반영

Git과 버전관리





Git의 탄생과 특징

□□꼴

놨놘놘뇔□꺼□□ □□□□꺼□□ □□□꿱□□

□□ □□꼴

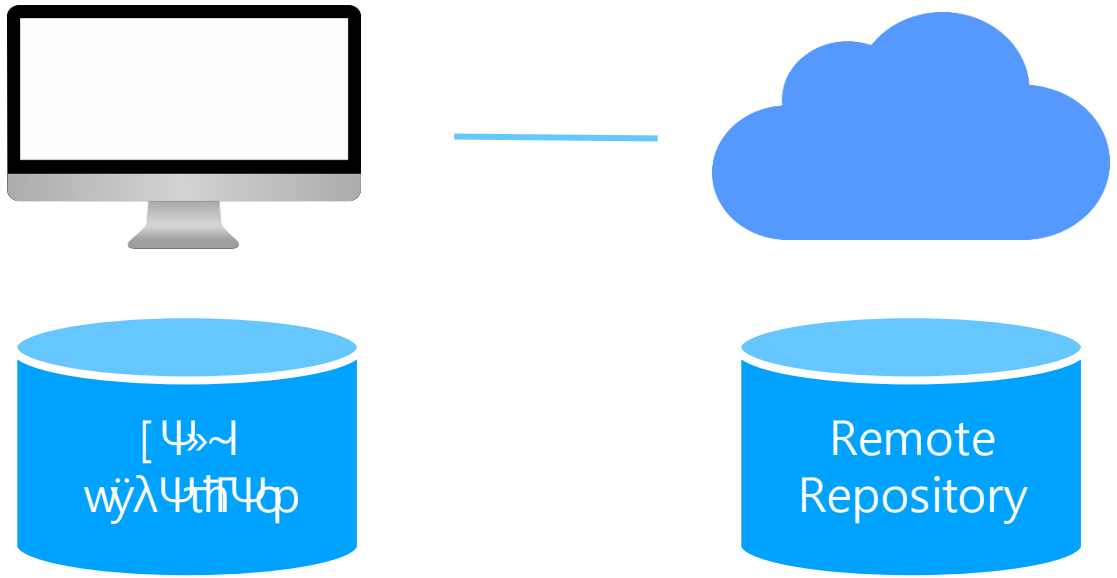
{ëb꿱Ⓐ□□□

로컬/원격저장소

로컬/원격저장소

로컬저장소와 원격저장소

- ✓ Git의 두 가지 저장소
- ✓ 로컬은 내 컴퓨터에서 관리
- ✓ 원격은 온라인에 공유



로컬/원격저장소

로컬저장소 Local Repository

내 컴퓨터에 코드와 이력 저장. Git은 저장할 공간만 있으면 어디서나 사용 가능(pc, usb, cloud 등)

□□□□ □□

```
git init
```

□□□□ □□ 꿸□□□ □□꿴

```
git add . or ${file name}
```

파일상태 확인

```
git status
```

변경사항 저장

```
git commit –m “${commit message}”
```

로컬/원격저장소

수정 → 준비 → 저장

01

ìΣιΑφΛý 6¤‘ ∝ψΣικ

파일 수정 단계

02

|ψ∽ý¤Λý ! ι‘ ∽

저장할 파일 선택

노팀

x‘ ζΣν¤ψΣικ

□□□□□

Add

/ΨΟΟ∩Τ

로컬/원격저장소

git init

ìΣιΑφΛý 6¤‘ ∝ψΣικ

|ψ∽ý¤Λý ! ι‘ ∽

x‘ ζΣν¤ψΣικ

로컬/원격저장소

ý¤ψ ¤Λ¤ψ

ìΣιΑ¤Λý 6¤‘ ∝ψΣικ

t ~ϋ놘놜

Page02

Page03

Page04

t ~ϋ놘뇔

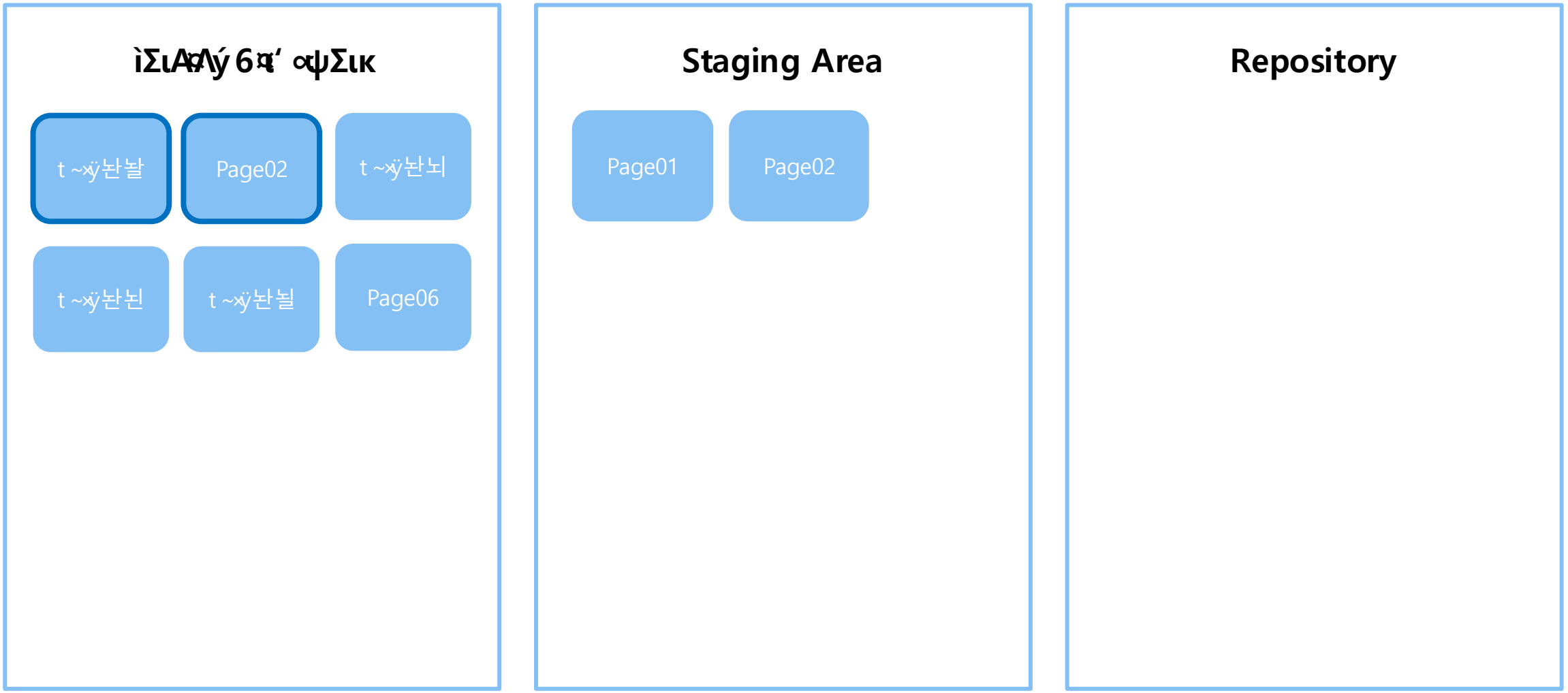
Page06

Staging Area

x‘ ζΣν¤ψΣικ

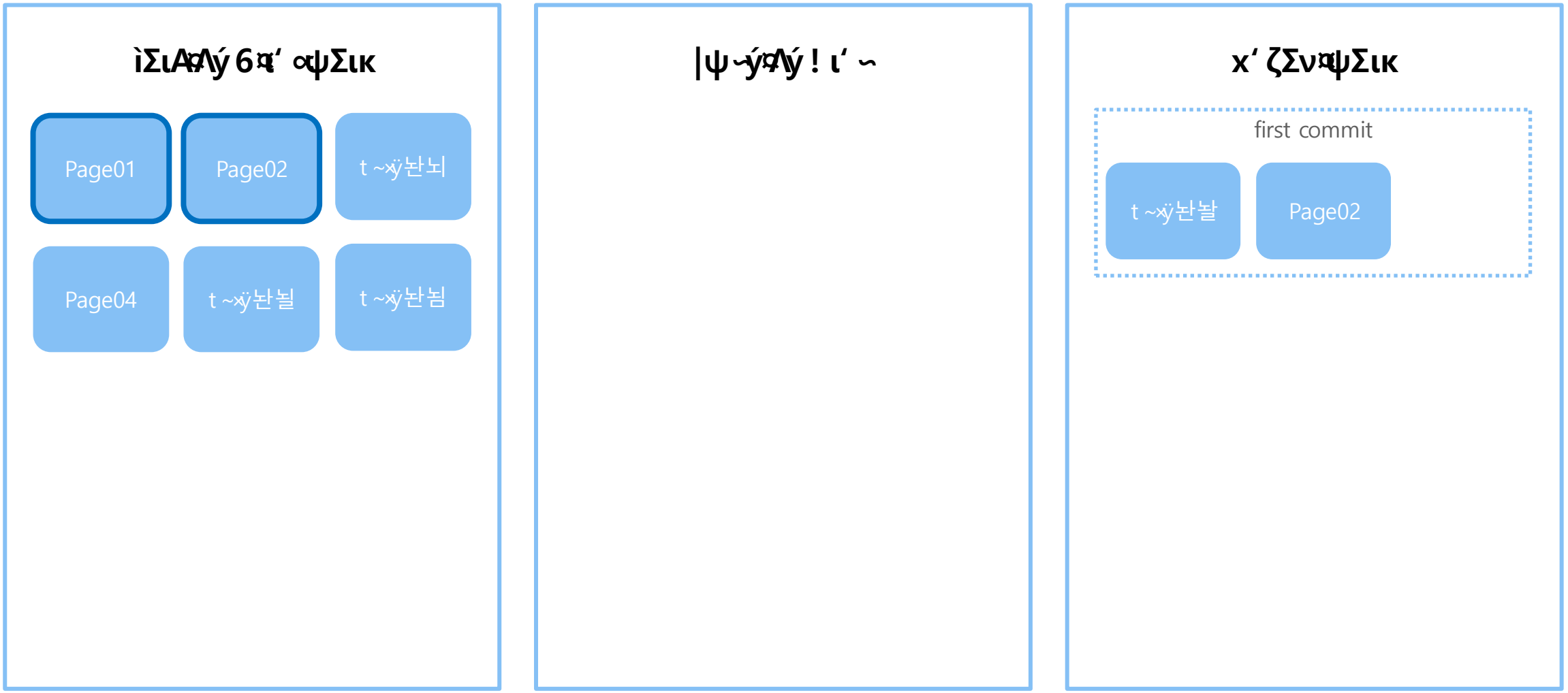
로컬/원격저장소

git add Page01 Page02



로컬/원격저장소

git commit –m “first commit”



로컬/원격저장소

ý¤ψ ~— 꼼

Working Directory

Page01

Page02

Page03

t ~ϰÿ놘뇐

Page05

t ~ϰÿ놘뇜

|ψ ~ý¤Λý ! ι' ~

Page03

Page04

t ~ϰÿ놘뇔

Page06

x' ζΣν¤ψΣικ

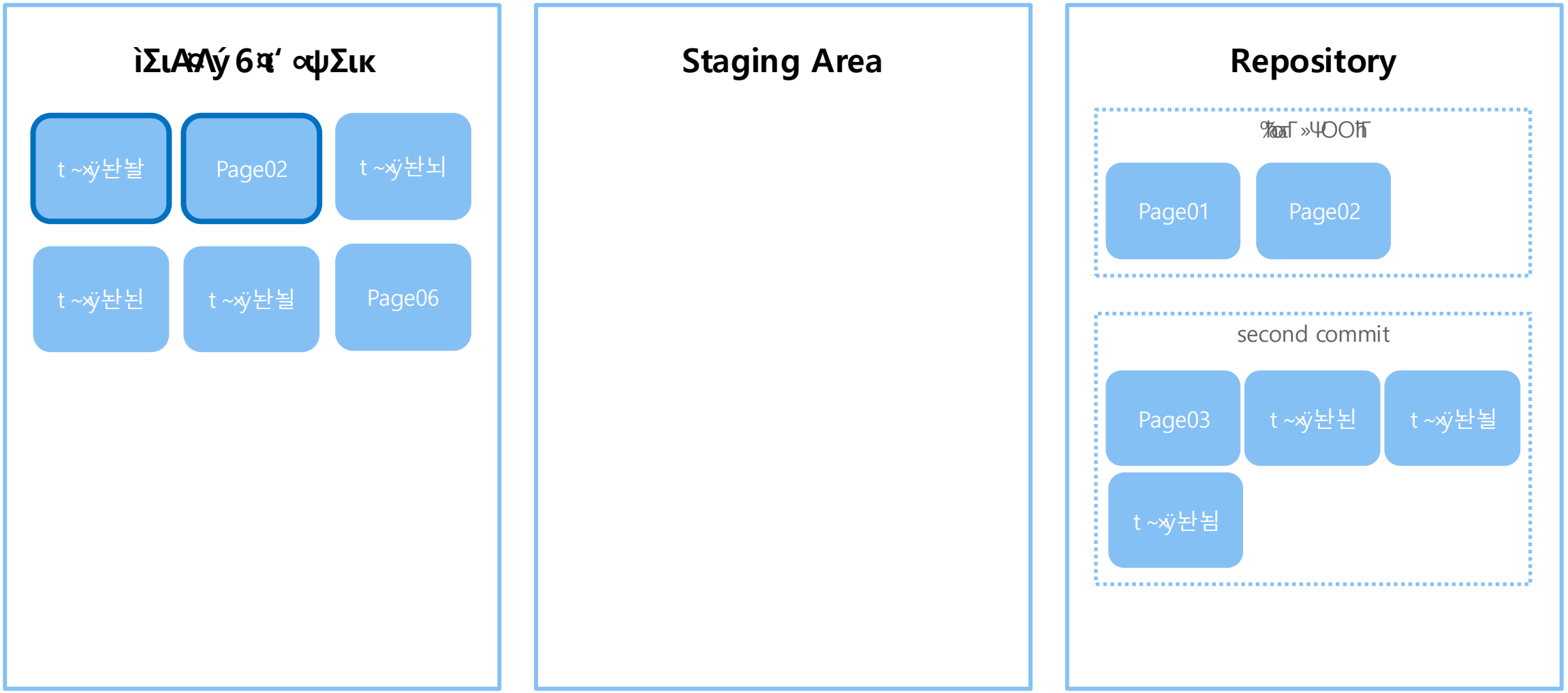
first commit

t ~ϰÿ놘놜

t ~ϰÿ놘놨

로컬/원격저장소

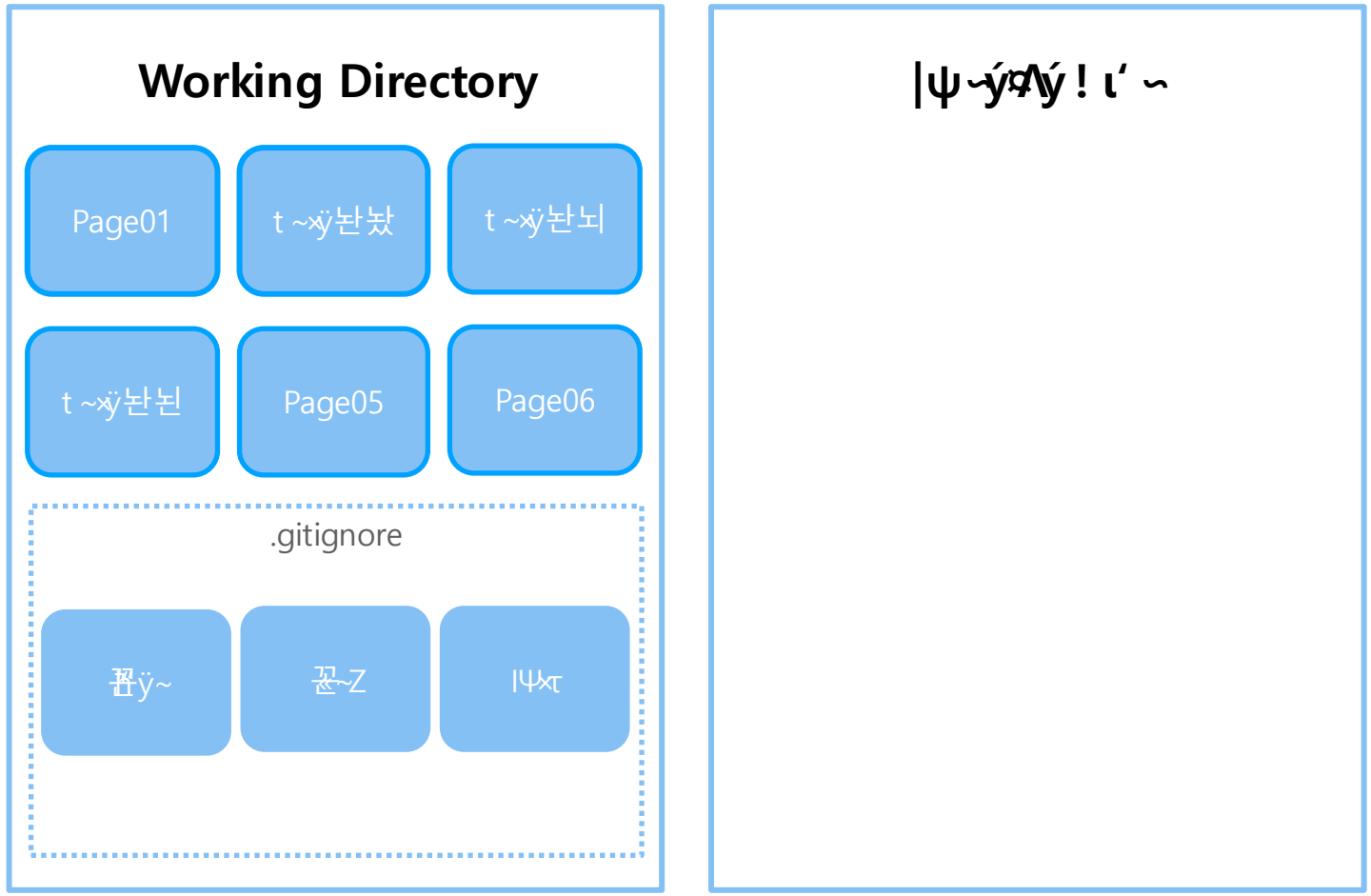
git commit –m “second commit”



로컬/원격저장소

□□ □□ 꿈ψ~ý^Σι'

- ✓ D㎧□ □□□□ □□ □□ □□ □□
- ✓ □□□□□□ □□□□깨□□□□ □□ □□! t L□□□ □ □ □□ □
- ✓ □□□□□□ □□□□ □□ □□ □□



로컬/원격저장소

원격저장소 Remote Repository

팀과 공유하며 어디서나 온라인으로 어디서나 접근 가능한 저장소

E¤ψ¯Ш√

GitLab

/¤ψ/Ш∘A' ψ

로컬/원격저장소

□□□□□x' KΣψ' x' ζΣνϖψΣικ

□□□□□ □□ 꿯∝~Δx' ζΣνψϖψΣικ□ □□ □꿤

```
ýϖψ ι' KΣψ' ~— 낭뀌ι' KΣψ' ι' ζΣ Λ~K' 꽃ιýϖΛ뀐 낭뀌ι' KΣψ' ι' ζΣνϖψΣικШιΔ뀐
```

원격저장소 확인

```
git remote -v
```

□□□□□ □□ 꿯∝~Δx' ζΣνϖψΣικ□ □□ □꿤

```
git clone ${remote repository url}
```

원격 저장소 파일 올리기

```
git push ${remote repo name : origin} ${branch name : main / dev ...}
```

□□ □□□ □□ □□□□ýϖψ ·' ψ∝¯ 댄 ýϖψ K' ιý'

```
git pull ${remote repo name : origin} ${branch name : main / dev ...}
```

로컬/원격저장소

A : git remote add origin {repositoryUrl}

ΨΣ∝⇝Δκ' ζΣνϖψΣικ !	Remote Repository	Local Repository B
%ΩΓ »ΨΟΟΠΓ		
τÿ»ΨΠ' »ΨΟΟΠΓ		

로컬/원격저장소

A : git push origin main

Ψ£∝⁄∆κ' ζΣνϖψΣικ !	Remote Repository	Local Repository B
%ϖαΓ »ΨΟΟ℘Γ	%ϖαΓ »ΨΟΟ℘Γ	
τÿ»ΨΠ' »ΨΟΟ℘Γ	second commit	

로컬/원격저장소

/ 꽃ψ ∝ΔΣΛ‘ 뀌ι‘ ζΣνψΣικάιΔ뀐

ΨΣ∝~Δκ‘ ζΣνψΣικ!

first commit

second commit

κ‘ KΣψ‘ κ‘ ζΣνψΣικ

%ΔΔΓ»ΨΟΟħΓ

second commit

Local Repository B

%ΔΔΓ»ΨΟΟħΓ

τÿ»ΨΠ‘ »ΨΟΟħΓ

로컬/원격저장소

B: git push origin main

Ψ£∝⇝∆κ' ζΣνϖψΣικ!

%ΩΓ »ΨΟΟħΓ

second commit

x' KΣψ' x' ζΣνϖψΣικ

first commit

second commit

new commit

Ψ£∝⇝∆κ' ζΣνϖψΣικ/

%ΩΓ »ΨΟΟħΓ

second commit

Γϊκ »ΨΟΟħΓ

로컬/원격저장소

A : git pull origin main

Local Repository A

%ΦΦΓ »ΨΟΟΦΓ

second commit

ΠΫκ »ΨΟΟΦΓ

x' KΣψ' x' ζΣνΦψΣικ

%ΦΦΓ »ΨΟΟΦΓ

τΫ»ΨΠ' »ΨΟΟΦΓ

new commit

Local Repository B

first commit

second commit

ΠΫκ »ΨΟΟΦΓ

Git 워크플로우

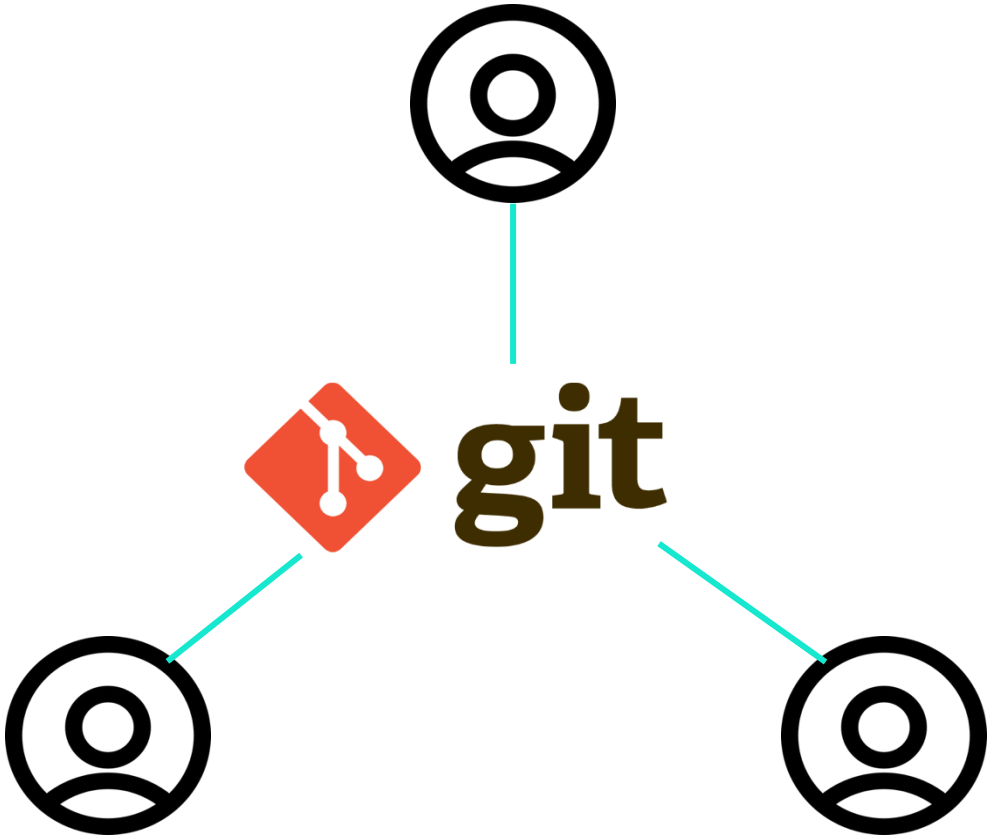
Git 워크플로우

Git Workflow

Git을 사용해 코드를 체계적으로 관리하는 규칙

워크플로우를 통해
작업 분리와 **체계적 병합**

- ✓ 변경 사항 추적
- ✓ 협업 효율성 증대
- ✓ 충돌 최소화



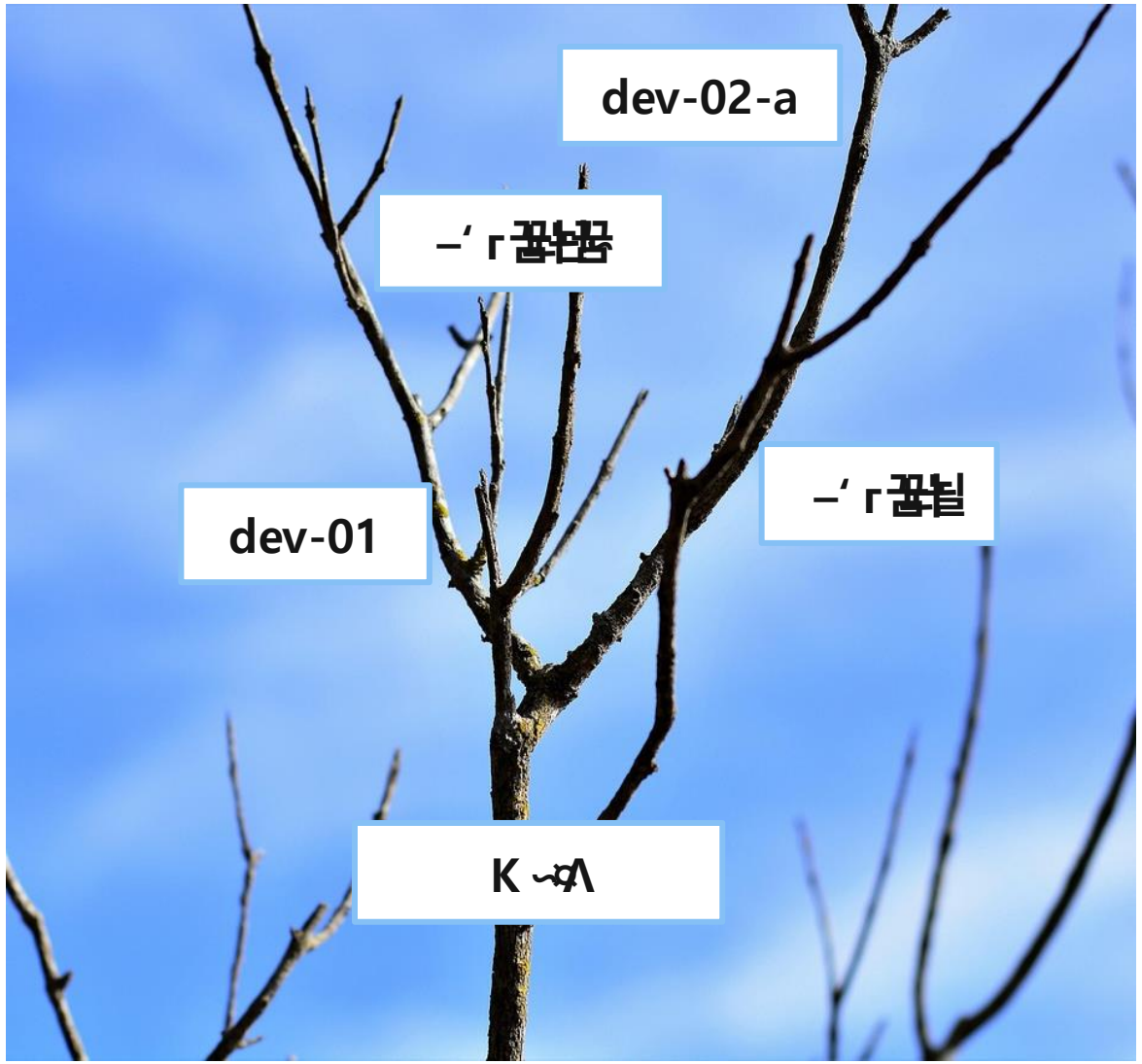
Git 워크플로우

브랜치 branch

작업 분리 공간.
기능별로 코드 분리 가능.

✓ 병렬 개발

✓ 실험 및 기능 추가



Git 워크플로우

□□□ □□ □ □□

□□꾀쌓ㅇㄱㅁ○ÿ 끊끊꾀□□꾀ÿlÿ~tÿ 끊끊꾀□□꾀ㅣㅓㅠ%ㅁ끊끊꾀 □□□ □□ □□□□ □□□ □□

브랜치 생성

```
ýœψ √ι∽Λ∝ 낭뀌√ι∽Λ∝ Λ∽K ' 뀐
```

```
git branch feature/login-new
```

브랜치 확인 (브랜치 목록 및 활성 브랜치 확인)

```
ýœψ √ι∽Λ∝
```

```
ýœψ √ι∽Λ∝
```

브랜치 전환

```
ýœψ ∝ ' ∘AΣШψ 낭뀌√ι∽Λ∝ Λ∽K ' 뀐
```

```
ýœψ ∝ ' ∘AΣШψ · ' ∽ψШι ' 꾀ýΛ꿈 ' д
```

브랜치 생성 및 전환

```
ýœψ ∝ ' ∘AΣШψ 꿯 낭뀌√ι∽Λ∝ Λ∽K ' 뀐
```

```
git checkout –b release/1.0.0
```

Git 워크플로우

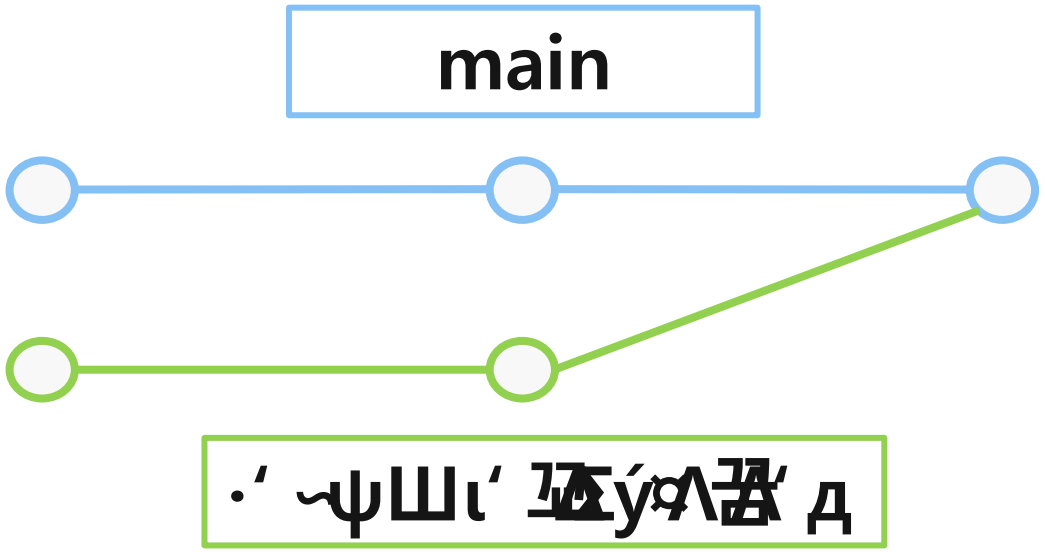
브랜치 병합

분기됐던 브랜치를 하나의 브랜치로 병합. **충돌** 발생 시 **수동으로 코드 수정** 후 병합 완료

□□□ □□

```
git merge ${branch name}
```

```
꾜ý¤ψ √ι∽Λ∝̄ □□ □□
ý¤ψ √ι∽Λ∝̄

꾜K ∽¤Λ √ι∽Λ∝̄ □ □□□
ý¤ψ ∝̄ ' ∘ΛΣШψ K ∽¤Λ

꾜‘ ∽ψШι‘ 꾜ý¤Λ器‘ д □□□□K ∽¤Λ □□□□ □□
ý¤ψ K ' ιý‘ ·‘ ∽ψШι‘ 꾜ý¤Λ器‘ д
```



Git 워크플로우

□□ □□

□□□□ □□□ □□□

O~ħ꾀□꾀ÿňÿ|ЧΛ꾀□ □□꾀
%ÿ∘ΓΩÿ꾐Γ«꾀□꾀ÿ|ÿ~τÿ꾐날꾼꾼꾀□ □□꾀ЧΓ%ω꾐ΓΩ꾼뇌뇔꾀□ □□꾀

단기간 유지

장기 브랜치는 병합 복잡도 증가

feature/gnb 브랜치와 feature/login 브랜치가 merge 없이 장기간 나눠져 있을 경우,
두 브랜치를 병합할 시점에 다양한 지점에서 충돌의 가능성이 높음

□□ □□□

□□ □□□□ □□ □□

□□□ □□□□ □□□ □□ □□□□ O~ħ□ □□□□ □□□□ □□□ □□ □□

Git 워크플로우

01

0' Λψι~Δ≠ί –
ìΣιΑ·Δξд

메인 브랜치 중심
소규모 팀에 적합

□□꽃□ □□□꺾⊕ ~ħ⊓ 꼼O ~τΓÿo꽥□

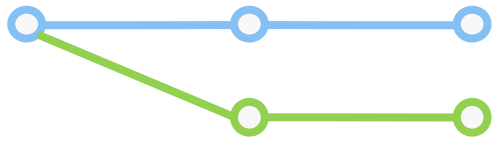
□□꽃□□꺼⊞□ □□

□□꽃□ □□꺼⊞□ □□□

O ~ħ⊓꺾⊞□꿱

Git 워크플로우

02



D ~ψШι' / ι~Λ∝̄
ìΣιA-Δд

기능별 브랜치 활용
코드 검토 강화

특징: **기능별 브랜치**(feature/*) 사용

장점: 코드 검토 용이, 협업 강화

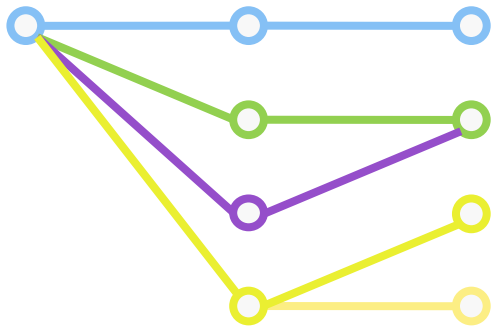
단점: 브랜치 관리 필요

main(배포)

∟ feature/*(기능)

Git 워크플로우

03



Gitflow Workflow

□□□꿰□□ □□□ □□
□□□ □□□□□ □□

특징: 복잡한 프로젝트 관리에 적합.

장점: 작업 분리와 **통합 체계화**.

단점: 초보자에겐 복잡, 팀 규칙 필요.

main(배포)

ㄴ release/*(배포 준비)

ㄴ hotfix/*(긴급 수정)

ㄴ develop(개발 통합)

ㄴ feature/*(기능)

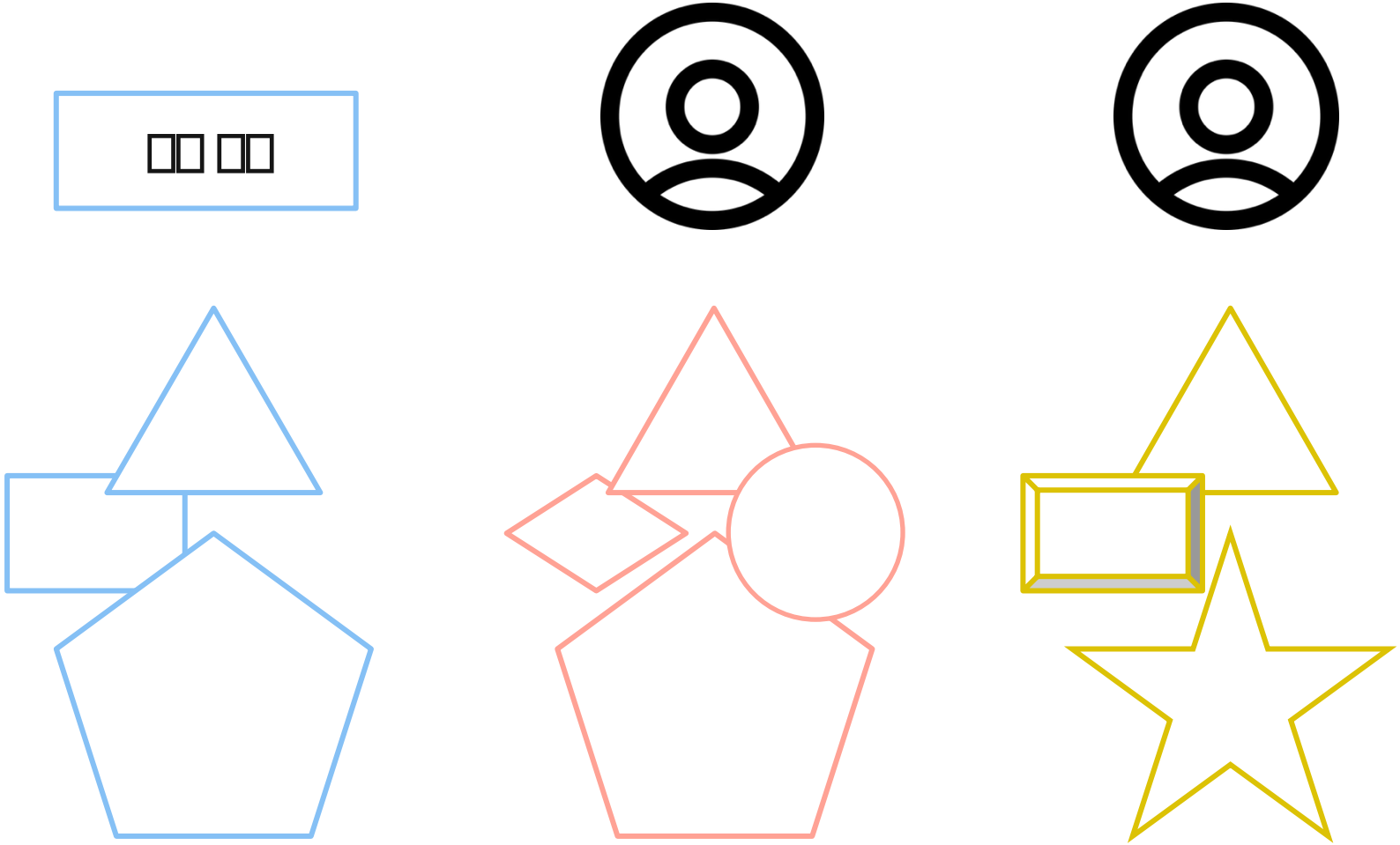
Git 워크플로우

브랜치 이름	역할	예시 사용 사례
main	배포용 최종 버전	제품 출시 후 안정 버전 관리
release/*	배포 준비	버전 1.0 배포 전 테스트
hotfix/*	긴급 수정	버그 긴급 패치
develop	개발 중 통합 버전	기능 병합 후 테스트
feature/*	새 기능 개발	로그인 기능 추가

Git 워크플로우

Conflict 충돌

동일 파일, 동일 라인 수정시
branch merge 충돌 발생



Git 워크플로우

□□ □□

□□ □□꿰ㅓㅓㅓ

Git 워크플로우

□□ □□

놜 꾼□ □□ □□ 꾱ý¤ψ νψ~ψШν □□

놨 꾼□ □□ 꾹□ □□□□ □□ □□□ □□ □□□ □□□□ □□□ □□□ □□

뇌 꾼□ □□ 꾹ㅊ ~" " 뉩□□ 꿥□ □□□□ □ý¤ψ ∝ΣΚ Κ ¤ψ □□ □□

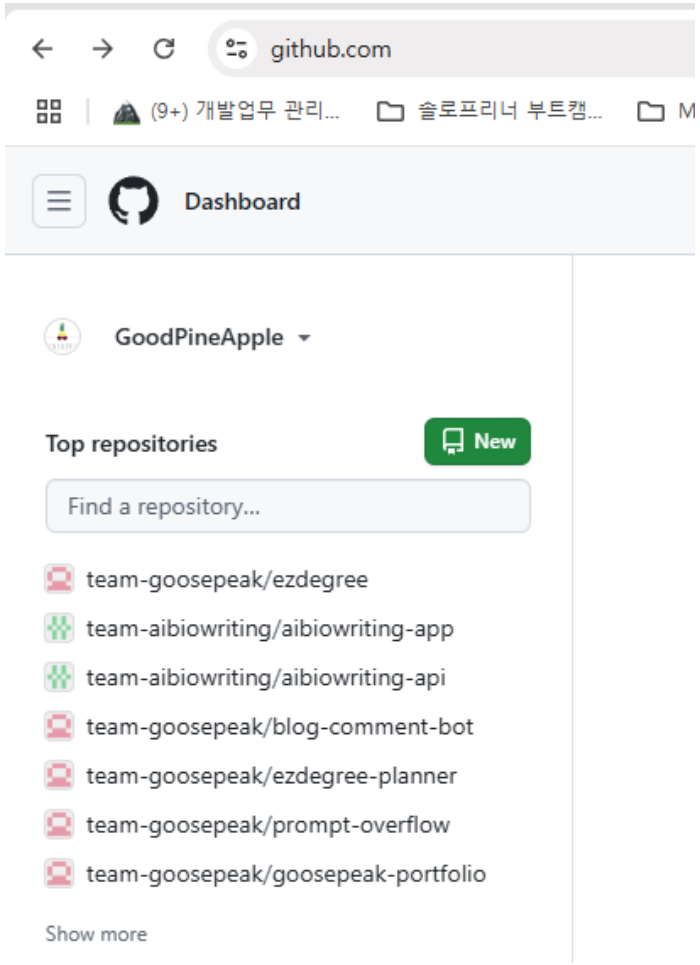
Oz-lecture

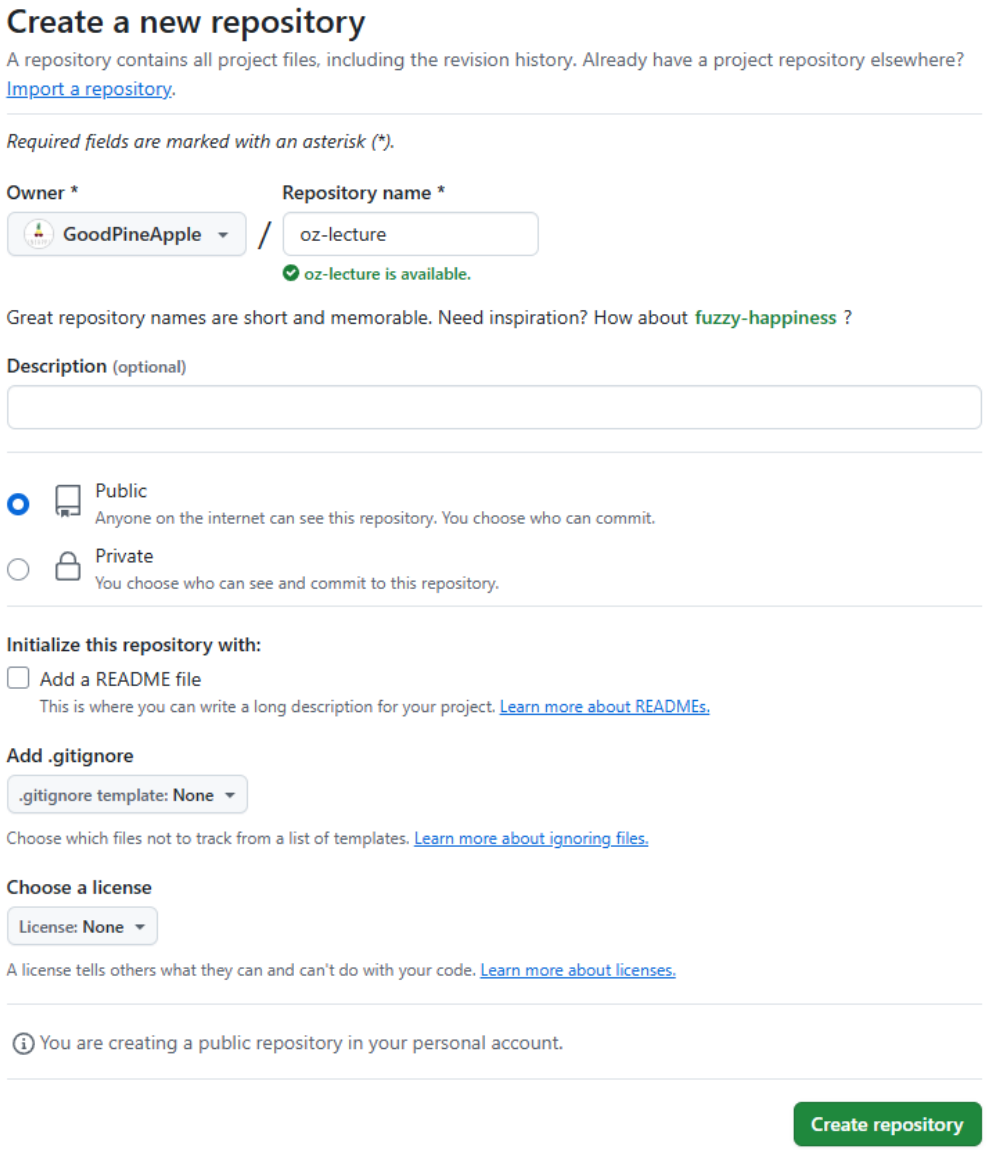
E¤ψ¯Ш√x‘ζΣν¤ψικ □□

□□□□ 꿷ÿκ 꿷

wÿλΨtñΨqp b ~Oÿ꼭
Ψx꾕ÿ»ΠΘÿ

□□□□ 꼭ΗΘlñ»
Dññ»ΠΨÿ꼭ΨΠÿ
[ñ»ÿΠtÿ꼭ΨΠÿ





₩Σ∝∽∆x' ζΣν¤ψικ □□

íν ∝Σ–' □□□ □□

í ħΠ'ψk 꼭⇒Γd 닿 X

a ~» 꼭⇒ψOO~Π' 닿 W

Eψ □□□

×ħΠ ħħħ

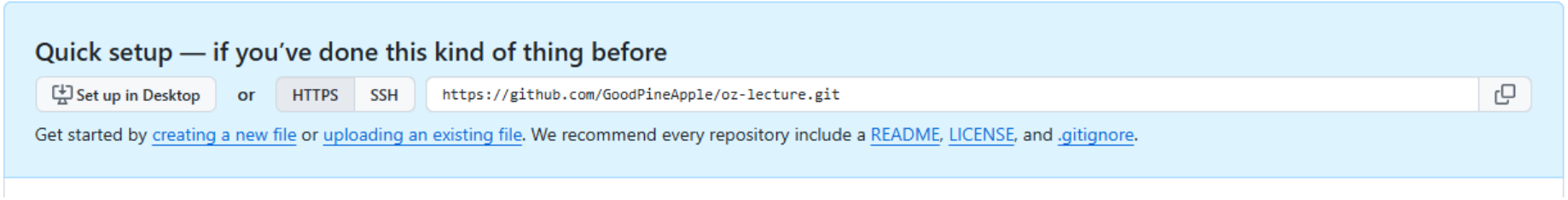
×ħΠ ~"" 꼰

×ħΠ »ψOOħΠ 꼰⇒ 꼰ħΠ λdψB'»Γ꼿

```
DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture
$ git init
Initialized empty Git repository in E:/98.developer-private/3.lecture/projects/oz-lecture/.git/

DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture (main)
$ git add .

DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture (main)
$ git commit -m "init project"
[main (root-commit) b35ecd4] init project
 14 files changed, 526 insertions(+)
 create mode 100644 day01/first.html
 create mode 100644 day01/second.html
 create mode 100644 day02/form.html
 create mode 100644 day02/multimedia.html
 create mode 100644 day02/semantic.html
 create mode 100644 day03/my-website.html
 create mode 100644 day03/styles.css
 create mode 100644 day04/my-view.html
 create mode 100644 day04/styles.css
 create mode 100644 day05/meida/index.html
 create mode 100644 day05/meida/styles-desktop-first.css
 create mode 100644 day05/meida/styles-mobile-first.css
 create mode 100644 day05/meida/styles.css
 create mode 100644 day07/bootstrap/index.html
```

x' ζΣν¤ψικ □□

E¤ψ¯Ш√x' ζΣν¤ψΣικ
D⌂ □□ □□

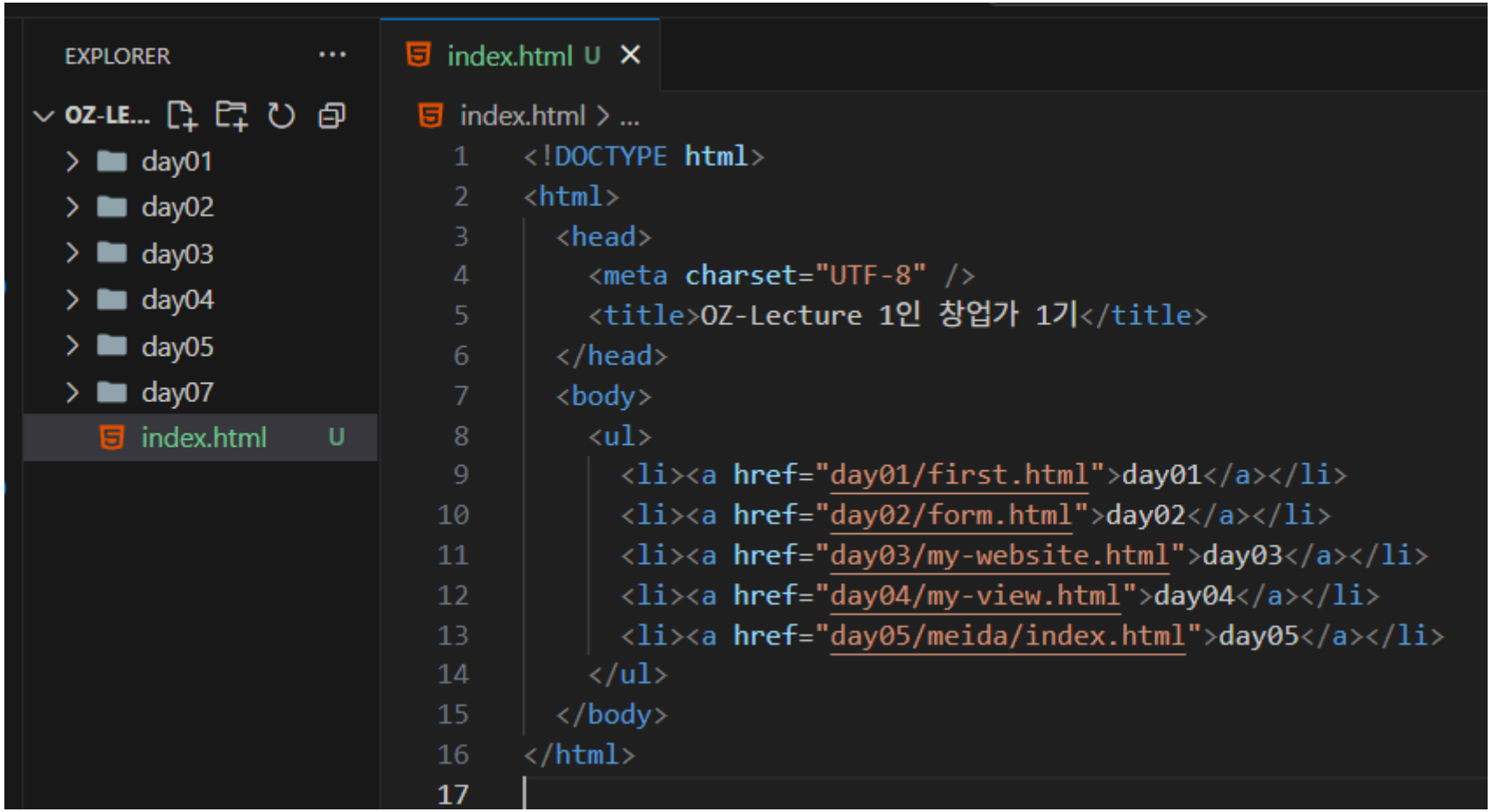


₩Ξ∝~Δx' ζΣν¤ψΣικ
×⌂ öÿΟ↑Γÿ ~" " ↑d⌂×⌂ 꿲⌂⁴ Η⊗ Η⊗ 꿲

×⌂ öÿΟ↑Γÿ 꿲

```
DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture (main)
$ git remote add origin https://github.com/GoodPineApple/oz-lecture.git

DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture (main)
$ git remote -v
origin  https://github.com/GoodPineApple/oz-lecture.git (fetch)
origin  https://github.com/GoodPineApple/oz-lecture.git (push)
```

index.html 작성



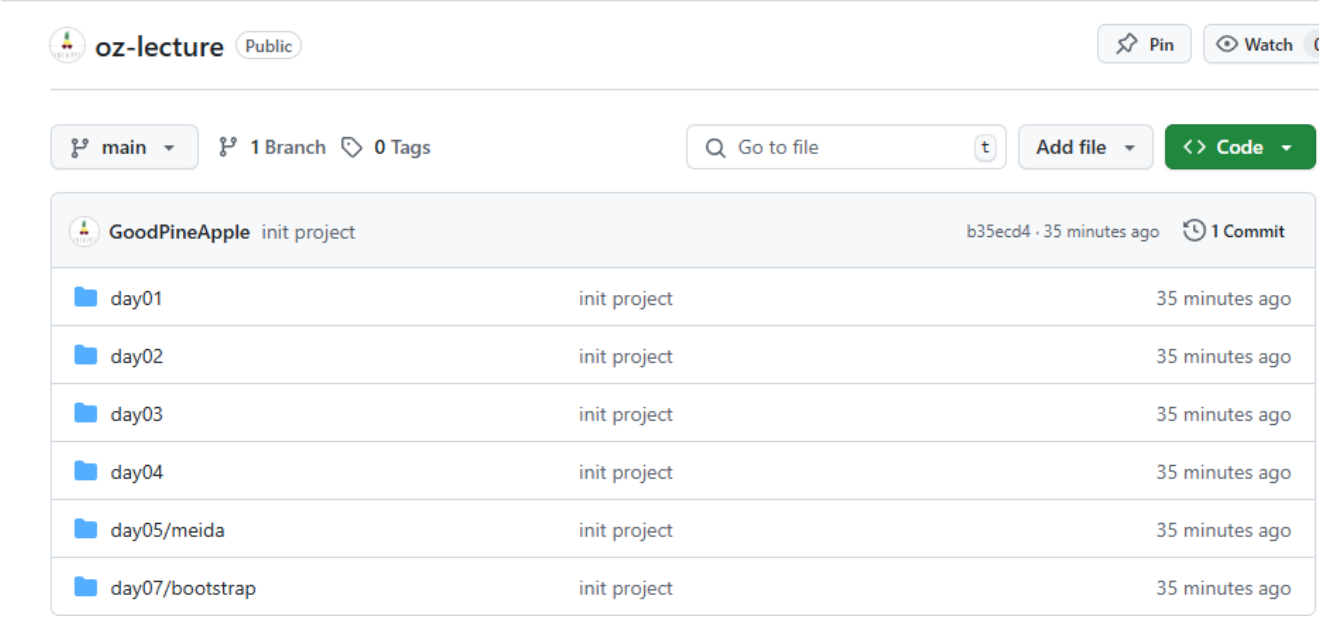
코드 push

Local

git push origin main

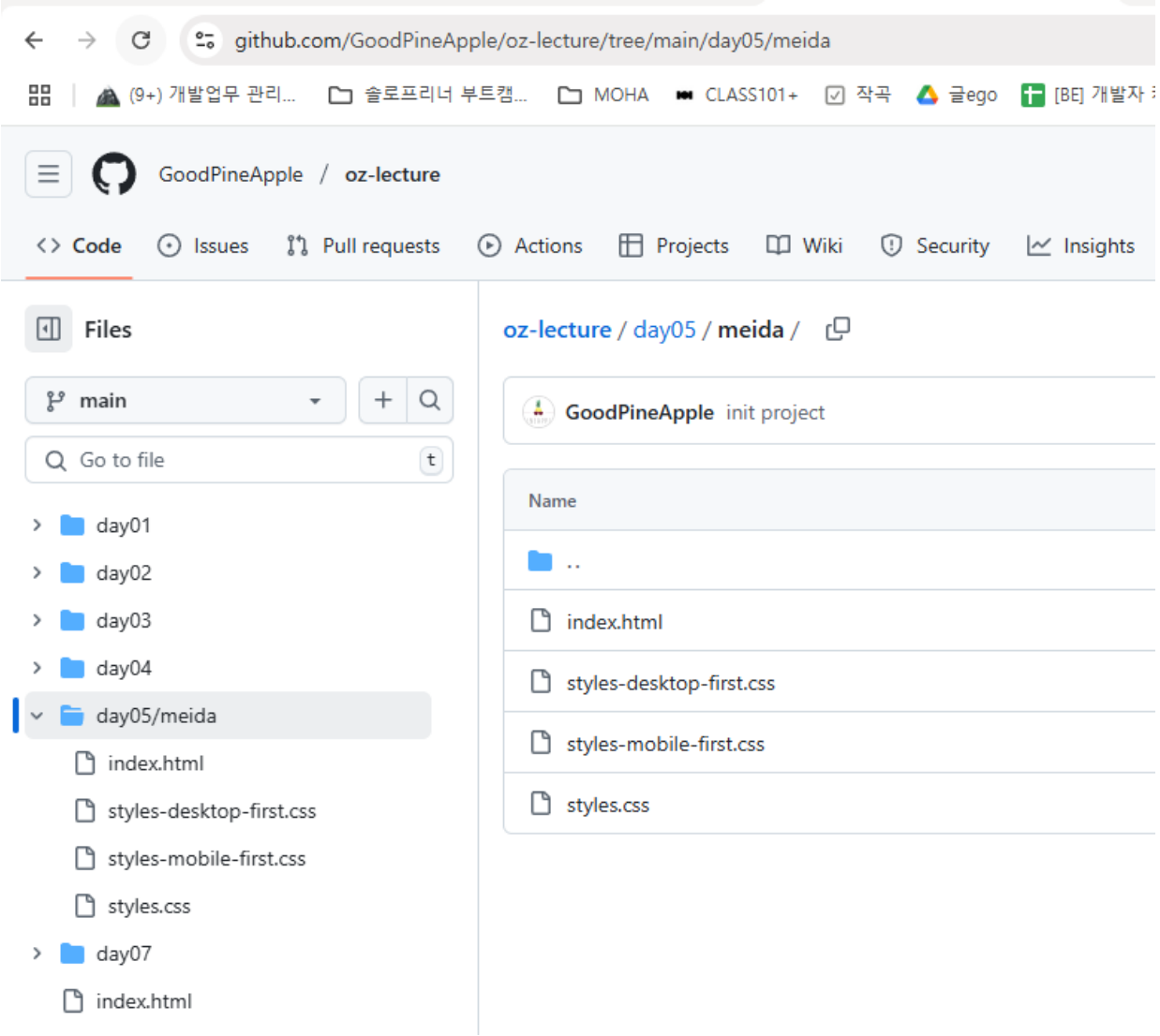
GitHub 변경 확인

```
DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture (main)
$ git push origin main
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 16 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (24/24), 6.77 KiB | 2.26 MiB/s, done.
Total 24 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/GoodPineApple/oz-lecture.git
 * [new branch]      main -> main
```



과제 제출

Github 링크
+
실행 캡쳐화면



오늘 우리는

Git과 버전관리

버전관리 Version Control System

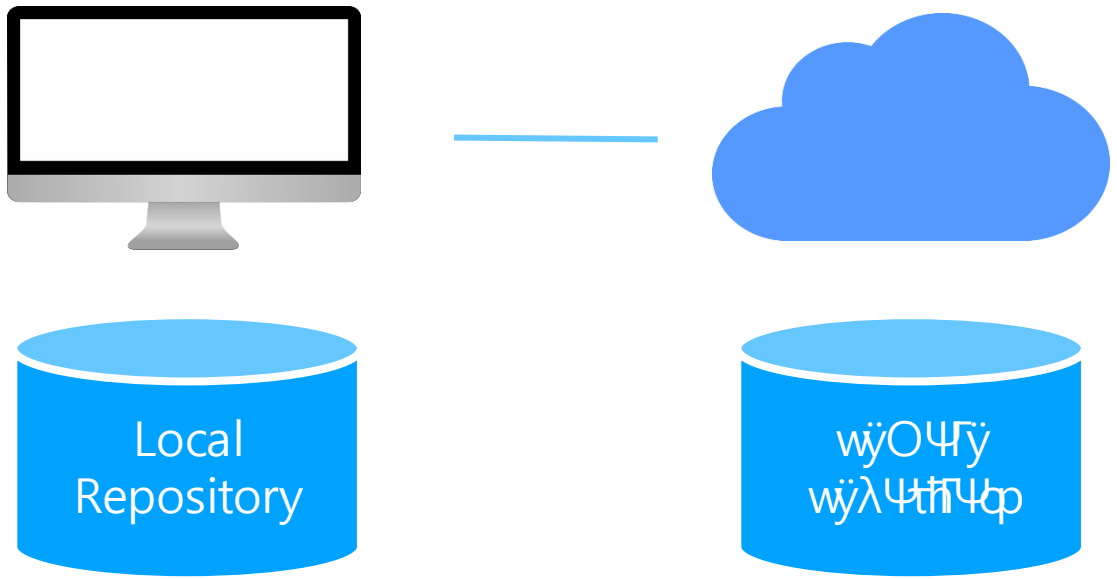
파일 변경 이력을 기록하고 관리하는 방법

1. 협업 : 여러 사람이 동시에 작업 가능
2. 복구 : 실수해도 이전 버전으로 돌아감
3. 추적 : 누가 언제 무엇을 바꿨는지 확인 가능

로컬/원격저장소

로컬저장소와 원격저장소

- ✓ Git의 두 가지 저장소
- ✓ 로컬은 내 컴퓨터에서 관리
- ✓ 원격은 온라인에 공유



로컬/원격저장소

원격저장소 Remote Repository

팀과 공유하며 어디서나 온라인으로 어디서나 접근 가능한 저장소

Github

E¤ψ₩~√

BitBucket

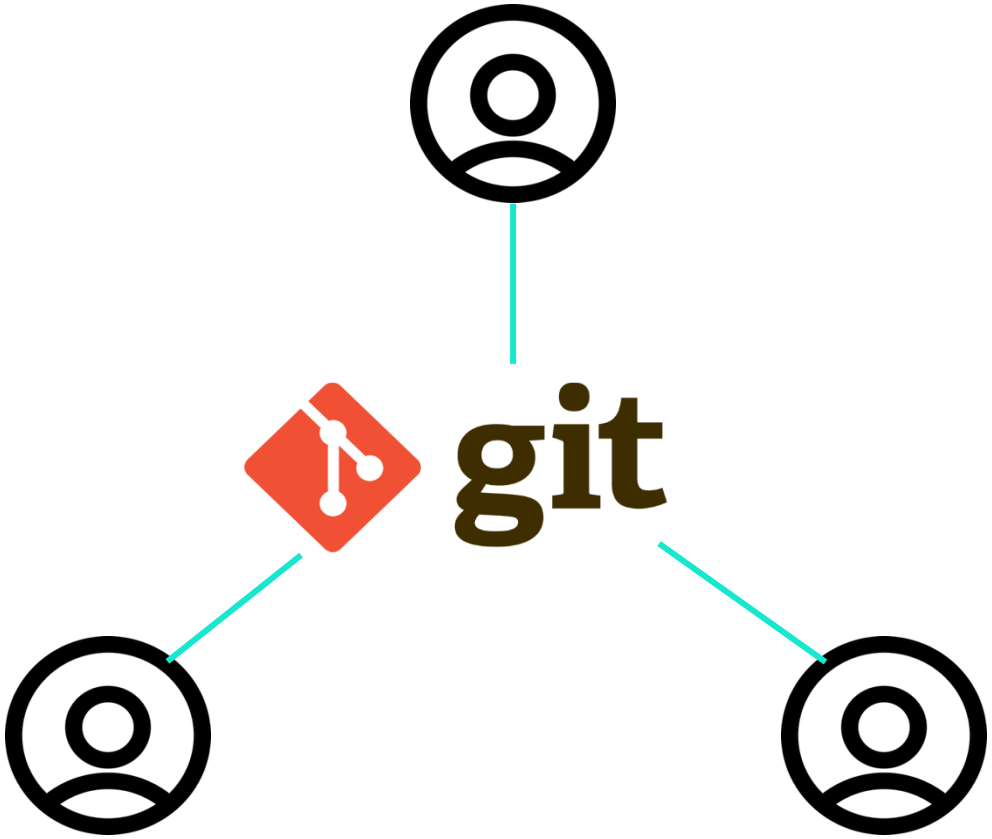
Git 워크플로우

Git Workflow

Git을 사용해 코드를 체계적으로 관리하는 규칙

워크플로우를 통해
작업 분리와 **체계적 병합**

- ✓ 변경 사항 추적
- ✓ 협업 효율성 증대
- ✓ 충돌 최소화



충돌 해결

1. 충돌 파일 확인 : **git status** 실행
2. 수동 수정 : 코드 편집기로 충돌 파일을 열어 **마커를 제거**하고 원하는 코드로 수정
3. 병합 완료 : git add ${파일명} 으로 스테이징 후 **git commit** 으로 커밋

감사합니다